

Linked List Patterns Guide

5 Most Important Linked List Patterns Prepared for
Interview Preparation & Smart Problem Solving

Most students solve Linked List questions randomly and get stuck in interviews. The real trick is to identify the **pattern** behind the problem. This guide contains:

- Pattern Name
- How to Identify the Pattern
- Core Intuition
- Frequently Used Logic
- Handpicked LeetCode Problems for Practice If you master these 5 patterns, you can solve most Linked List interview questions confidently.

1. Basic Operations Pattern

How to Identify This Pattern?

- Question asks to insert/delete/traverse nodes
- Need direct pointer manipulation
- Head node changes frequently
- Mostly uses temp/current/prev pointers

Core Intuition

This pattern is all about understanding how nodes are connected using next pointers. You should become comfortable changing links without losing references.

Important Logic to Remember

- Always save next node before changing links
- Use dummy node when head modifications are involved
- Handle edge cases like empty list or single node

Practice Problems

- LeetCode 203 – Remove Linked List Elements

- LeetCode 206 – Reverse Linked List
 - LeetCode 707 – Design Linked List
 - LeetCode 83 – Remove Duplicates from Sorted List
-

2. Fast & Slow Pointer Pattern

How to Identify This Pattern?

- Need middle node
- Need cycle detection
- Question mentions loop/circular structure
- Need to find nth node from end efficiently

Core Intuition

Use two pointers moving at different speeds. Fast pointer usually moves 2 steps while slow pointer moves 1 step. Their meeting point reveals important information.

Important Logic to Remember

- Fast = 2 steps, Slow = 1 step
- If fast reaches null → no cycle
- If fast meets slow → cycle exists

Practice Problems

- LeetCode 141 – Linked List Cycle
 - LeetCode 142 – Linked List Cycle II
 - LeetCode 876 – Middle of the Linked List
 - LeetCode 19 – Remove Nth Node From End
-

3. Reversal Pattern

How to Identify This Pattern?

- Question asks to reverse entire list

- Reverse between positions
- Reverse in k-groups
- Need backward traversal behavior

Core Intuition

The main idea is changing direction of links. Instead of node pointing forward, it points backward.

Important Logic to Remember

- Maintain prev, curr, next
- Never lose next node reference
- Practice dry runs carefully

Practice Problems

- LeetCode 206 – Reverse Linked List
 - LeetCode 92 – Reverse Linked List II
 - LeetCode 25 – Reverse Nodes in K Group
 - LeetCode 234 – Palindrome Linked List
-

4. Merge / Sort Pattern

How to Identify This Pattern?

- Two sorted linked lists are given
- Need sorting
- Question asks to reorder nodes
- Need divide and conquer logic

Core Intuition

This pattern combines merging logic with pointer manipulation. Merge Sort is heavily used because random access is expensive in linked lists.

Important Logic to Remember

- Use dummy node while merging

- Find middle using slow-fast pointers
- Recursively split and merge

Practice Problems

- LeetCode 21 – Merge Two Sorted Lists
 - LeetCode 148 – Sort List
 - LeetCode 143 – Reorder List
 - LeetCode 23 – Merge K Sorted Lists
-

5. Linked List + Stack Pattern

How to Identify This Pattern?

- Need backward traversal
- Need next greater node
- Carry propagation problems
- Cannot traverse backward directly

Core Intuition

Linked Lists only move forward, but stacks help simulate reverse traversal behavior.

Important Logic to Remember

- Push nodes/values into stack
- Pop elements to process in reverse order
- Useful for palindrome and next greater problems

Practice Problems

- LeetCode 1019 – Next Greater Node In Linked List
 - LeetCode 445 – Add Two Numbers II
 - LeetCode 234 – Palindrome Linked List
 - LeetCode 496 – Next Greater Element I
-

Final Advice:

Do not solve Linked List questions randomly. First identify the pattern, then think about the logic template. When you repeatedly practice pattern-wise problems, your brain automatically starts recognizing approaches during interviews. That is the real goal of DSA preparation.